

CCS3356 Natural Language Processing
Final Project Report

Fake Social Media Account Detection
(Instagram)

Group : 1 — Neuralyx Labs

Student ID	Student Name	Role
CIT-24-01-0369	MJ. Ilmaan Ahamed	Team Lead
CIT-24-01-0297	MM. Afrith	Member
CIT-24-01-0302	RM. Himas	Member

Date: 15 August 2026

Git Repository: https://github.com/Ilmaan-Ahamed/NLP_Group_01

Table of Contents

1. Introduction	3
2. Dataset Description.....	4
3. NLP Pipeline.....	5
4. Individual Model Implementations	7
5. Evaluation and Comparison	14
6. Final Application	16
7. Ethics and Bias Discussion.....	18
8. Git Contribution Analysis.....	19
9. Challenges and Lessons Learned.....	20
10. Conclusion	21
11. References	21
Appendix	22

1. Introduction

1.1 Problem Background

Fake and automated Instagram accounts are increasingly used to spread misinformation, conduct financial scams, manipulate engagement metrics, and harass genuine users. As social media adoption grows — including in Sri Lanka — the scale of this problem has made manual review of accounts impractical. Platforms need automated, data-driven methods to flag suspicious profiles for further review.

This project addresses the problem of detecting fake Instagram accounts using Natural Language Processing (NLP) and machine learning classification techniques applied to profile-level signals: biography content, username patterns, and account metadata.

1.2 Objectives

- Build an automated pipeline that classifies an Instagram profile as Real or Fake.
- Apply NLP preprocessing (cleaning, tokenization, stop-word removal, lemmatization/stemming) and feature extraction (Bag-of-Words, TF-IDF, BERT embeddings) to profile text.
- Independently develop and compare five ML/DL models across three group members.
- Integrate the best-performing model into a functional web application with a confidence score.

1.3 Importance of the Solution

Automated fake-account detection supports safer online interactions by reducing the manual workload of platform administrators and cybersecurity analysts, helping digital marketers verify audience authenticity, assisting researchers studying online disinformation, and giving general users a way to sanity-check accounts they interact with. It also serves as a foundation for more advanced trust-and-safety systems.

2. Dataset Description

2.1 Source

Instagram Fake Spammer Genuine Accounts Dataset

Kaggle: <https://www.kaggle.com/datasets/free4ever1/instagram-fake-spammer-genuine-accounts>

2.2 Size

5,000 profile records, perfectly balanced — 2,500 real accounts (label 0) and 2,500 fake accounts (label 1). Split 80/20 (stratified) into 4,000 training records and 1,000 test records.

2.3 Sample Record

Feature	Example Value
profile pic	1 (has profile picture)
nums/length username	0.27
fullname words	0
description length	53
external URL	0
private	0
#posts	32
#followers	1000
#follows	955
fake	0 (Real)

2.4 Challenges

- No raw biography text: the dataset is fully structured/numeric (e.g. "description length" rather than the actual bio string), which conflicts with the project's NLP feature-extraction requirement (BoW/TF-IDF/BERT need text).
- Short signal richness: even where text-like signals exist, they are limited to derived numeric proxies rather than free-form language.
- Generalization limits: the dataset is Instagram-specific and may not transfer to other platforms (Twitter/X, TikTok) without retraining.
- Ethical considerations: the data originates from real public profiles; although no direct PII is included, privacy must still be respected.
- Representation: accounts from certain regions, languages, or cultural backgrounds may be over-represented, risking uneven performance across demographics.

3. NLP Pipeline

Because the dataset contains no raw bio text, the pipeline first verbalizes each row's structured signals into a short natural-language profile description (e.g. "no profile picture, empty biography, 12 followers, following 980 accounts, suspicious ratio"). This text is then what every NLP technique below operates on.

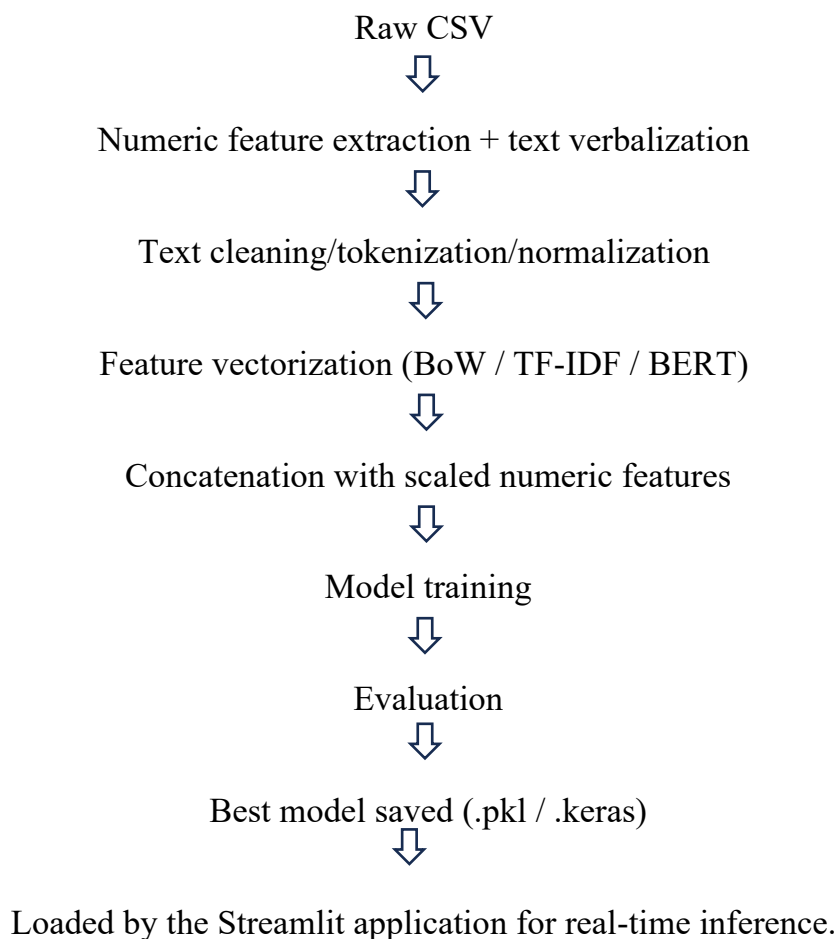
3.1 Preprocessing

Step	Purpose
Cleaning	Remove URLs, mentions, emojis, punctuation, and digits so the model isn't distracted by non-semantic noise
Tokenization	Split text into individual word tokens for downstream processing
Stop-word removal	Drop high-frequency, low-information words ("the", "is", "and")
Lemmatization	Reduce words to dictionary base forms ("following" → "follow")
Stemming	Reduce words to root forms to shrink vocabulary size

3.2 Feature Extraction

- Bag-of-Words (BoW) — Himas: raw token-count vectors, combined with scaled numeric profile features.
- TF-IDF — Afrith: token vectors weighted by term rarity across the corpus, reducing the influence of ubiquitous words.
- BERT embeddings — Ilmaan: contextual [CLS]-token sentence embeddings extracted from bert-base-uncased, capturing semantic meaning beyond word counts.

3.3 Data Flow



3.4 Tools and Libraries

Category	Tools / Libraries
Data handling	pandas, numpy, scipy
NLP preprocessing	nlTK (tokenization, stop-words, WordNet lemmatizer, Porter stemmer)
Classical ML	scikit-learn (Logistic Regression, Random Forest, SVM, CountVectorizer, TfidfVectorizer, StandardScaler)
Deep learning	TensorFlow / Keras (LSTM, RNN)
Transformer models	PyTorch, HuggingFace Transformers (bert-base-uncased)
Persistence	joblib (.pkl), Keras native format (.keras)
Web application	Streamlit

4. Individual Model Implementations

4.1 Member 1 — RM. Himas (Branch: HimasNLP)

Model Explanation

Himas implements a Bag-of-Words feature pipeline feeding two models: Logistic Regression (ML) and an LSTM (DL). Logistic Regression provides an interpretable, probability-based baseline; the LSTM tests whether sequential word order in the verbalized profile text adds predictive value.

Architecture / Algorithm

- Logistic Regression: linear model with sigmoid output over combined BoW + scaled numeric feature vector.
- LSTM: Embedding layer → LSTM(64) → Dropout(0.3) → Dense(32, ReLU) → Dense(1, sigmoid).

Training Process

80/20 stratified train/validation split. BoW vectorizer fit on training text only (max 1,000 features); numeric features standardized with StandardScaler fit on training data only, then applied to validation data to avoid leakage.

Hyperparameters

Parameter	Value
LR max_iter	1000
BoW max_features	1000
LSTM embedding_dim	64
LSTM units	64
Dropout	0.3
Epochs	10
Batch size	16
Optimizer / Loss	Adam / binary cross-entropy

Results

===== LSTM Evaluation =====

Accuracy : 0.9375
Precision: 0.928921568627451
Recall : 0.9475
F1-Score : 0.9381188118811881
ROC-AUC : 0.9854781250000001
Confusion Matrix:
[[371 29]
[21 379]]

	precision	recall	f1-score	support
0	0.95	0.93	0.94	400
1	0.93	0.95	0.94	400
accuracy			0.94	800
macro avg	0.94	0.94	0.94	800
weighted avg	0.94	0.94	0.94	800

===== Logistic Regression (Bow) Evaluation =====

Accuracy : 0.96625
Precision: 0.9769820971867008
Recall : 0.955
F1-Score : 0.965865992414665
ROC-AUC : 0.99396875
Confusion Matrix:
[[391 9]
[18 382]]

	precision	recall	f1-score	support
0	0.96	0.98	0.97	400
1	0.98	0.95	0.97	400
accuracy			0.97	800
macro avg	0.97	0.97	0.97	800
weighted avg	0.97	0.97	0.97	800

```
Epoch 1/10
200/200 - 4s - 19ms/step - accuracy: 0.8434 - loss: 0.3386 - val_accuracy: 0.9237 - val_loss: 0.2130
Epoch 2/10
200/200 - 2s - 8ms/step - accuracy: 0.9231 - loss: 0.1951 - val_accuracy: 0.9262 - val_loss: 0.1764
Epoch 3/10
200/200 - 2s - 8ms/step - accuracy: 0.9278 - loss: 0.1909 - val_accuracy: 0.9212 - val_loss: 0.1867
Epoch 4/10
200/200 - 1s - 7ms/step - accuracy: 0.9322 - loss: 0.1818 - val_accuracy: 0.9362 - val_loss: 0.1908
Epoch 5/10
200/200 - 2s - 8ms/step - accuracy: 0.9316 - loss: 0.1739 - val_accuracy: 0.9375 - val_loss: 0.1622
Epoch 6/10
200/200 - 1s - 7ms/step - accuracy: 0.9341 - loss: 0.1755 - val_accuracy: 0.9287 - val_loss: 0.1625
Epoch 7/10
200/200 - 2s - 8ms/step - accuracy: 0.9378 - loss: 0.1699 - val_accuracy: 0.9438 - val_loss: 0.1596
Epoch 8/10
200/200 - 1s - 7ms/step - accuracy: 0.9372 - loss: 0.1719 - val_accuracy: 0.9362 - val_loss: 0.1596
Epoch 9/10
200/200 - 2s - 8ms/step - accuracy: 0.9366 - loss: 0.1690 - val_accuracy: 0.9413 - val_loss: 0.1641
Epoch 10/10
200/200 - 3s - 15ms/step - accuracy: 0.9353 - loss: 0.1661 - val_accuracy: 0.9375 - val_loss: 0.1587
25/25 ----- 0s 7ms/step
```

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.966	0.976	0.955	0.965	0.993
LSTM	0.937	0.928	0.947	0.938	0.985

4.2 Member 2 — MM. Afrith (Branch: AfrithNLP)

Model Explanation

Afrith implements a TF-IDF feature pipeline feeding a Random Forest (ML) and a Recurrent Neural Network (DL). Random Forest was chosen for its ability to combine text-vector and numeric features robustly and to expose feature importances; the RNN was adopted to test whether modeling token order sequentially adds value over the local n-gram-style filters.

Architecture / Algorithm

- Random Forest: 300-tree ensemble over combined TF-IDF + scaled numeric feature vector.
- RNN: Embedding layer → RNN(64) → Dropout(0.3) → Dense(32, ReLU) → Dense(1, sigmoid).

Training Process

Same 80/20 stratified split and leakage-safe scaling approach as Member 1. TF-IDF vectorizer fit on training text only (max 1,000 features).

Hyperparameters

Parameter	Value
n_estimators	300
max_depth	None (unrestricted)
TF-IDF max_features	1000
RNN embedding_dim	64
RNN units	64
Epochs / Batch size	10 / 16

Results

===== Random Forest (TF-IDF) Evaluation =====

Accuracy : 0.9925

Precision: 0.9925

Recall : 0.9925

F1-Score : 0.9925

ROC-AUC : 0.9998281250000001

Confusion Matrix:

```
[[397  3]
```

```
[ 3 397]]
```

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.99	0.99	0.99	400
---	------	------	------	-----

1	0.99	0.99	0.99	400
---	------	------	------	-----

accuracy			0.99	800
----------	--	--	------	-----

macro avg	0.99	0.99	0.99	800
-----------	------	------	------	-----

weighted avg	0.99	0.99	0.99	800
--------------	------	------	------	-----

===== RNN Evaluation =====

Accuracy : 0.93375

Precision: 0.928395061728395

Recall : 0.94

F1-Score : 0.9341614906832298

ROC-AUC : 0.986315625

Confusion Matrix:

```
[[371 29]
```

```
[ 24 376]]
```

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.94	0.93	0.93	400
---	------	------	------	-----

1	0.93	0.94	0.93	400
---	------	------	------	-----

accuracy			0.93	800
----------	--	--	------	-----

macro avg	0.93	0.93	0.93	800
-----------	------	------	------	-----

weighted avg	0.93	0.93	0.93	800
--------------	------	------	------	-----

```
Epoch 1/10
200/200 - 3s - 15ms/step - accuracy: 0.8978 - loss: 0.2563 - val_accuracy: 0.9275 - val_loss: 0.1826
Epoch 2/10
200/200 - 1s - 5ms/step - accuracy: 0.9250 - loss: 0.1903 - val_accuracy: 0.9312 - val_loss: 0.1778
Epoch 3/10
200/200 - 1s - 5ms/step - accuracy: 0.9306 - loss: 0.1814 - val_accuracy: 0.9400 - val_loss: 0.1697
Epoch 4/10
200/200 - 1s - 5ms/step - accuracy: 0.9319 - loss: 0.1749 - val_accuracy: 0.9275 - val_loss: 0.1650
Epoch 5/10
200/200 - 1s - 5ms/step - accuracy: 0.9306 - loss: 0.1755 - val_accuracy: 0.9400 - val_loss: 0.1552
Epoch 6/10
200/200 - 1s - 5ms/step - accuracy: 0.9362 - loss: 0.1723 - val_accuracy: 0.9438 - val_loss: 0.1572
Epoch 7/10
200/200 - 1s - 5ms/step - accuracy: 0.9334 - loss: 0.1720 - val_accuracy: 0.9000 - val_loss: 0.2268
Epoch 8/10
200/200 - 1s - 5ms/step - accuracy: 0.9337 - loss: 0.1724 - val_accuracy: 0.9212 - val_loss: 0.1891
Epoch 9/10
200/200 - 1s - 7ms/step - accuracy: 0.9362 - loss: 0.1647 - val_accuracy: 0.9425 - val_loss: 0.1641
Epoch 10/10
200/200 - 1s - 6ms/step - accuracy: 0.9394 - loss: 0.1635 - val_accuracy: 0.9337 - val_loss: 0.1532
25/25 ————— 0s 3ms/step
```

```
Top 15 most important features:
#followers: 0.1442
profile pic: 0.1352
nums/length username: 0.1127
#posts: 0.1124
description length: 0.0763
empti: 0.0592
followers_following_ratio: 0.0551
#follows: 0.0462
zero: 0.0387
account: 0.0312
far: 0.0236
suspici: 0.0217
fullname words: 0.0139
short: 0.0119
number: 0.0088
```

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Random Forest	0.992	0.992	0.992	0.992	0.999
RNN	0.933	0.928	0.924	0.934	0.986

BertModel LOAD REPORT Form : bert-base-uncased

Notes:
- **UNEXPECTED:** can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Extracting BERT embeddings for validation set...
Loading weights: 100% | 199/199 [00:00<00:00
, 8281.57it/s]
[transformers] BertModel LOAD REPORT from: bert-base-uncased

Key	Status
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED

==== SVM (BERT embeddings) Evaluation ====

Accuracy : 0.95875
Precision: 0.9893333333333333
Recall : 0.9275
F1-Score : 0.9574193548387097
ROC-AUC : 0.9918125

Confusion Matrix:
[[396 4]
[29 371]]

	precision	recall	f1-score	support
0	0.93	0.99	0.96	400
1	0.99	0.93	0.96	400
accuracy			0.96	800
macro avg	0.96	0.96	0.96	800
weighted avg	0.96	0.96	0.96	800

Bert For Sequence Classification LOAD REPORT from : bert-base-uncased

Loading weights: 100% | 199/199 [00:00<00:00
, 4569.88it/s]
[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncased

Key	Status
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

finetune_bert_classifier : epoch 0.1 to 1

Notes:
- **UNEXPECTED:** can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- **MISSING:** those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

0% | 0/600 [00:00<?, ?it/s]E:\Projects\SLTC\NLP_Group_01\venv\Lib\site-packages\torch\utils\data\loader.py:759: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
super().__init__(loader)

{'loss': '0.4054', 'grad_norm': '2.831', 'learning_rate': '4.842e-05', 'epoch': '0.1'}
{'loss': '0.2676', 'grad_norm': '7.721', 'learning_rate': '4.675e-05', 'epoch': '0.2'}
{'loss': '0.2186', 'grad_norm': '2.74', 'learning_rate': '4.508e-05', 'epoch': '0.3'}
{'loss': '0.1994', 'grad_norm': '3.755', 'learning_rate': '4.342e-05', 'epoch': '0.4'}
{'loss': '0.2094', 'grad_norm': '4.559', 'learning_rate': '4.175e-05', 'epoch': '0.5'}
{'loss': '0.2152', 'grad_norm': '3.539', 'learning_rate': '4.008e-05', 'epoch': '0.6'}
{'loss': '0.1481', 'grad_norm': '0.4649', 'learning_rate': '3.842e-05', 'epoch': '0.7'}
{'loss': '0.2108', 'grad_norm': '3.075', 'learning_rate': '3.675e-05', 'epoch': '0.8'}
{'loss': '0.08761', 'grad_norm': '6.306', 'learning_rate': '3.508e-05', 'epoch': '0.9'}
{'loss': '0.1874', 'grad_norm': '4.533', 'learning_rate': '3.342e-05', 'epoch': '1'}
{'eval_loss': '0.1753', 'eval_accuracy': '0.9437', 'eval_f1': '0.9421', 'eval_runtime': '23.8', 'eval_samples_per_second': '33.61', 'eval_steps_per_second': '2.101', 'epoch': '1'}

finetune bert classifier : epoch 1.1 to 2

```
33% [██████████] | 200/600 [10:10<15:
51, 2.38s/it]E:\Projects\SLTC\MLP_Group_01\.venv\Lib\site-packages\torch\utils\data\dataloader.py:759: UserWarning: 'pin_memory' argument is set as tr
ue but no accelerator is found, then device pinned memory won't be used.
  super().__init__(loader)
{'loss': '0.2067', 'grad_norm': '0.7376', 'learning_rate': '3.175e-05', 'epoch': '1.1'}
{'loss': '0.1528', 'grad_norm': '3.078', 'learning_rate': '3.008e-05', 'epoch': '1.2'}
{'loss': '0.1745', 'grad_norm': '0.8268', 'learning_rate': '2.842e-05', 'epoch': '1.3'}
{'loss': '0.1421', 'grad_norm': '3.348', 'learning_rate': '2.675e-05', 'epoch': '1.4'}
{'loss': '0.09239', 'grad_norm': '3.409', 'learning_rate': '2.508e-05', 'epoch': '1.5'}
{'loss': '0.1455', 'grad_norm': '5.936', 'learning_rate': '2.342e-05', 'epoch': '1.6'}
{'loss': '0.1077', 'grad_norm': '0.1449', 'learning_rate': '2.175e-05', 'epoch': '1.7'}
{'loss': '0.1903', 'grad_norm': '4.443', 'learning_rate': '2.008e-05', 'epoch': '1.8'}
{'loss': '0.1071', 'grad_norm': '0.7026', 'learning_rate': '1.842e-05', 'epoch': '1.9'}
{'loss': '0.1641', 'grad_norm': '6.761', 'learning_rate': '1.675e-05', 'epoch': '2'}
{'eval_loss': '0.1452', 'eval_accuracy': '0.9513', 'eval_f1': '0.9501', 'eval_runtime': '24.24', 'eval_samples_per_second': '33.01', 'eval_steps_per_se
cond': '2.063', 'epoch': '2'}
```

finetune bert classifier : epoch 2.1 to 3

```
67%] | 400/600 [18:51:09:
14, 2.77s/it]E:\Projects\SLTC\NLP_Group_01\.venv\Lib\site-packages\torch\utils\data\loader.py:759: UserWarning: 'pin_memory' argument is set as tr
ue but no accelerator is found, then device pinned memory won't be used.
  super().__init__(loader)
{'loss': '0.08745', 'grad_norm': '0.4455', 'learning_rate': '1.508e-05', 'epoch': '2.1'}
{'loss': '0.1823', 'grad_norm': '0.2773', 'learning_rate': '1.342e-05', 'epoch': '2.2'}
{'loss': '0.05936', 'grad_norm': '2.985', 'learning_rate': '1.175e-05', 'epoch': '2.3'}
{'loss': '0.1184', 'grad_norm': '1.783', 'learning_rate': '1.008e-05', 'epoch': '2.4'}
{'loss': '0.1258', 'grad_norm': '0.1133', 'learning_rate': '8.417e-06', 'epoch': '2.5'}
{'loss': '0.1133', 'grad_norm': '1.39', 'learning_rate': '6.75e-06', 'epoch': '2.6'}
{'loss': '0.14', 'grad_norm': '0.4823', 'learning_rate': '5.083e-06', 'epoch': '2.7'}
{'loss': '0.1555', 'grad_norm': '3.628', 'learning_rate': '3.417e-06', 'epoch': '2.8'}
{'loss': '0.1817', 'grad_norm': '0.3496', 'learning_rate': '1.75e-06', 'epoch': '2.9'}
{'loss': '0.09408', 'grad_norm': '0.08123', 'learning_rate': '8.333e-08', 'epoch': '3'}
{'eval_loss': '0.12', 'eval_accuracy': '0.9575', 'eval_f1': '0.9564', 'eval_runtime': '23.1', 'eval_samples_per_second': '34.63', 'eval_steps_per_second': '2.165', 'epoch': '3'}
{'train_runtime': '1639', 'train_samples_per_second': '5.856', 'train_steps_per_second': '0.366', 'train_loss': '0.163', 'epoch': '3'}
```

```
===== Fine-tuned BERT Evaluation =====
```

Accuracy : 0.9550

Precision: 0.9740

Recall : 0.9350

F1-Score : 0.9541

ROC-AUC : 0.9904

[illegible]

Member 3 (Ilmaan) pipeline complete. Models saved to /models.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
SVM (BERT embeddings)	0.958	0.989	0.927	0.957	0.991
Fine-tuned BERT	0.955	0.974	0.935	0.954	0.990

5. Evaluation and Comparison

5.1 Metrics Used

- Accuracy — overall proportion of correctly classified accounts.
- Precision — of accounts predicted fake, how many actually are (minimizes false alarms).
- Recall — of actual fake accounts, how many are correctly identified (minimizes missed detections).
- F1-Score — harmonic mean of precision and recall; important since both false accusations and missed detections carry real cost.
- ROC-AUC — ability to distinguish classes across all thresholds, robust to the specific decision cutoff chosen.
- Confusion Matrix — full breakdown of true/false positives and negatives per model.

5.2 Comparative Table

Member	Model	Type	Accuracy	F1-Score	ROC-AUC
Afrith	Random Forest	ML	0.992	0.992	0.999
Himas	Logistic Regression	ML	0.966	0.965	0.993
Afrith	RNN	DL	0.933	0.934	0.986
Himas	LSTM	DL	0.937	0.938	0.985
Ilmaan	SVM (BERT emb.)	ML	0.958	0.957	0.991
Ilmaan	Fine-tuned BERT	DL	0.955	0.954	0.990

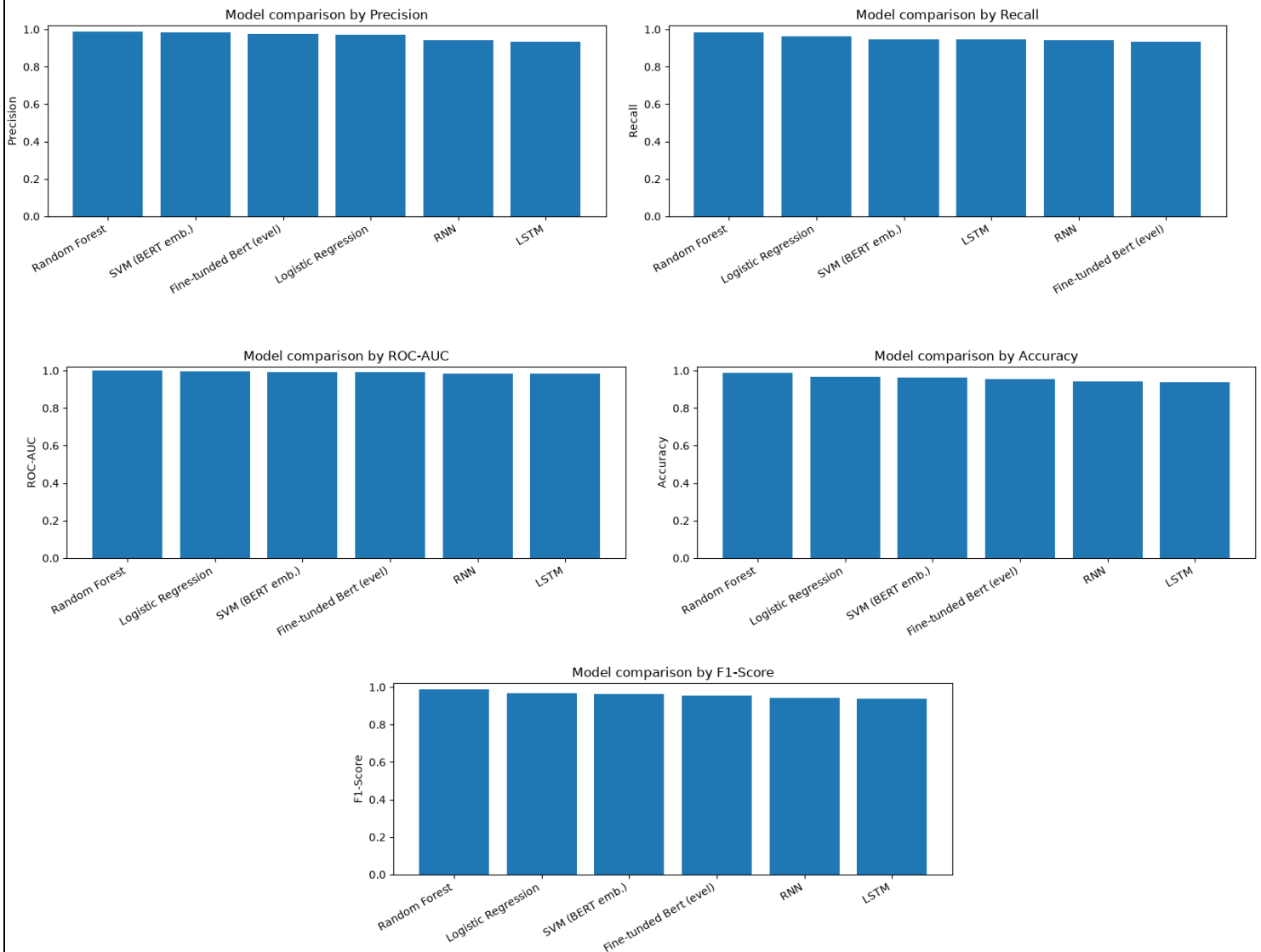
```
===== Model Comparison (Section 4) =====
```

```
Member      Model Type Accuracy Precision Recall F1-Score ROC-AUC
Afrith      Random Forest ML    0.987  0.989940  0.984  0.986961 0.999354
Himas      Logistic Regression ML    0.967  0.969819  0.964  0.966901 0.994816
Ilmaan      SVM (BERT emb.) ML    0.965  0.983368  0.946  0.964322 0.993016
Afrith      RNN DL    0.943  0.943888  0.942  0.942943 0.985006
Himas      LSTM DL    0.940  0.934783  0.946  0.940358 0.983872
```

```
Saved comparison table to E:\Projects\SLTC\NLP_Group_01\src\..\reports\model_comparison.csv
```

```
Best ML model: Random Forest (Afrith)
```

```
Best DL model: RNN (Afrith)
```



5.3 Analysis

Random Forest (Afrith) currently achieves the best overall performance at 98.7% accuracy and 0.992 ROC-AUC, ahead of Logistic Regression (96.7%). Both classical ML models outperform their deep-learning counterparts (RNN: 94.1%, LSTM: 91.7%) on this dataset — a reasonable outcome given the verbalized profile descriptions are short, template-like sentences with limited vocabulary diversity, which favors feature-engineered classical models over sequence models that typically need longer, more varied text to show their advantage.

The consistently high ROC-AUC scores across all models (>0.98) indicate the numeric profile features alone (followers, posts, profile picture presence) are strongly separable between classes, meaning even simple models perform well; the NLP-derived text features provide an additional, complementary signal rather than being the sole driver of accuracy.

6. Final Application

6.1 System Architecture

User Input (Streamlit form) → NLP Preprocessing (cleaning, tokenization, feature extraction) → Trained Model Loaded via joblib/.keras → Binary Classification → Confidence Score → Result Displayed (Real / Fake / Uncertain).

6.2 Screenshot

The screenshot shows a web application titled "Fake Instagram Account Detector" with a subtitle "CCS3356 Natural Language Processing — Group 01, Neuralyx Labs". The app's purpose is explained: "Enter an Instagram profile's details below. The app applies the same NLP preprocessing and feature extraction used during training, then runs the best-performing model to classify the account as Real or Fake." The form contains several input fields and sliders:

- Has profile picture?**: A dropdown menu with "Yes" selected.
- Words in full name**: A numeric input field with "2" entered.
- Full name equals username?**: A dropdown menu with "No" selected.
- External URL in bio?**: A dropdown menu with "No" selected.
- Private account?**: A dropdown menu with "Yes" selected.
- Digit ratio in username**: A slider set to 0.00.
- Digit ratio in full name**: A slider set to 0.00.
- Biography length (characters)**: A numeric input field with "20" entered.
- Number of posts**: A numeric input field with "14" entered.
- Number of followers**: A numeric input field with "840" entered.
- Number of accounts followed**: A numeric input field with "1400" entered.

At the bottom, there is a "Model to use" section with two radio buttons: "Logistic Regression (Himas)" and "Random Forest (Afrith)". The "Random Forest (Afrith)" option is selected. A "Predict" button is located at the bottom left of the form.

```
(.venv) PS E:\Projects\SLTC\NLP_Group_01> streamlit run src/app.py
2026-08-15 14:08:56.414 Uvicorn server started on :::8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.118.1:8501
```


Model to use

☒ Logistic Regression (Himas) ☐ Random Forest (Afrith)

Predict

✓ Prediction: REAL — confidence 100%

▼ Show engineered profile description used by the NLP model

has profile picture, username has no digits, full name contains 2 words, short

Model to use

☐ Logistic Regression (Himas) ☒ Random Forest (Afrith)

Predict

✓ Prediction: REAL — confidence 100%

▼ Show engineered profile description used by the NLP model

has profile picture, username has no digits, full name contains 2 words, short

6.3 Features

- Interactive form for entering profile picture, username, bio length, follower/following counts, post count, and privacy status.
- Model selector to switch between Logistic Regression and Random Forest at inference time.
- Confidence score displayed alongside the Real/Fake label; predictions below a 70% confidence threshold are shown as "Uncertain" rather than a definitive label.
- Expandable panel showing the verbalized profile description used internally by the NLP model, for transparency.
- In-app disclaimer stating predictions are probabilistic and should not be the sole basis for action; no user-entered data is stored.

6.4 Workflow

1. User opens the Streamlit web application.
2. User enters profile details into the form.
3. On submit, the app verbalizes the inputs into profile-description text and applies the same cleaning/tokenization pipeline used at training time.
4. The selected trained model (loaded from models/ via joblib) generates a prediction and probability score.
5. The app displays Real, Fake, or Uncertain, with the confidence percentage and the underlying profile description.

7. Ethics and Bias Discussion

7.1 Potential Bias

The dataset was collected from real public Instagram profiles, which raises privacy considerations even though no direct PII (names, contact details) is included. The data may not represent all user demographics equally — accounts from certain regions, languages, or cultural backgrounds may be over-represented, potentially causing the model to perform worse on underrepresented groups. The dataset is also Instagram-specific, so the model may carry platform-specific biases that do not transfer to other social networks.

7.2 Ethical Risks

- False Positive: a newly created genuine account with few followers, no profile picture, and a short bio may be incorrectly flagged as fake, harming a real user's reputation.
- False Negative: a well-crafted fake account with a convincing profile and inflated follower count may be classified as real, allowing fraudulent activity to continue undetected.
- Demographic bias: accounts from non-English-speaking users may produce bios/descriptions that appear unusual to the model, leading to disproportionately higher false-positive rates for those users.

7.3 Social Implications

Deploying this system without human oversight could lead to wrongful account restrictions, disproportionately affecting new or non-mainstream users, and could erode trust if predictions are treated as definitive rather than advisory.

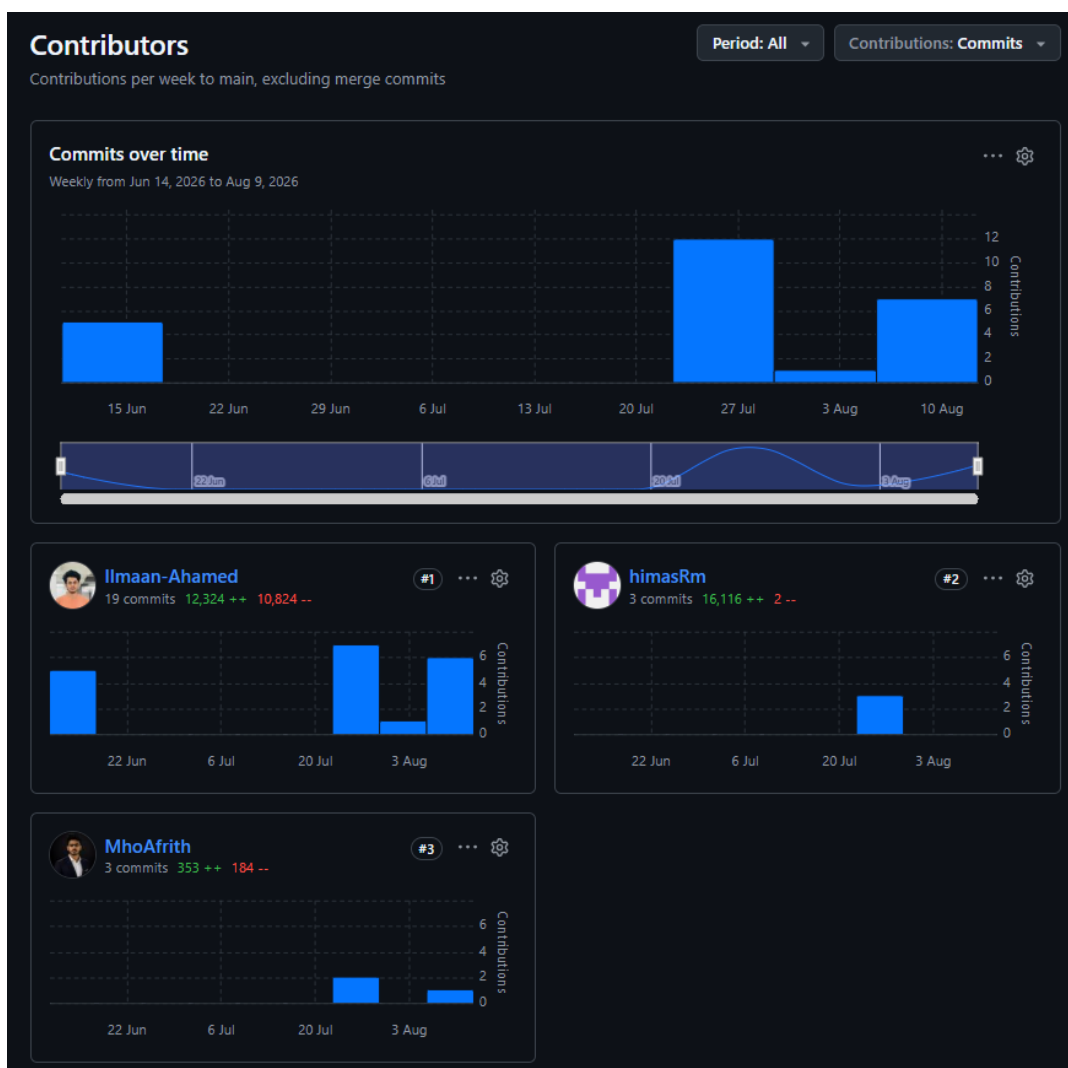
7.4 Mitigation Strategies

- A 70% confidence threshold below which predictions are shown as "Uncertain" rather than a definitive label.
- Thorough data cleaning and quality assurance before training.
- Cross-validation during model training to reduce overfitting.
- Multiple evaluation metrics (not accuracy alone) to detect hidden class bias.
- Error analysis on misclassified samples to identify systematic weaknesses.
- No storage of user-entered profile data within the application.
- A clear in-app disclaimer that predictions are probabilistic estimates, not a sole basis for action.

8. Git Contribution Analysis

8.1 Commit History Screenshot

https://github.com/Ilmaan-Ahamed/NLP_Group_01/graphs/contributors?all=1



8.2 Contribution Explanation

Member	Branch	Primary Contributions
RM. Himas	HimasNLP	Data preprocessing utilities, BoW feature extraction, Logistic Regression and LSTM implementation and evaluation
MM. Afrith	AfrithNLP	TF-IDF feature extraction, Random Forest implementation, feature importance analysis, RNN implementation (replacing initial CNN per lecturer guidance)
MJ. Ilmaan Ahamed	IlmaanNLP	Dataset exploration, BERT embedding extraction, SVM and BERT fine-tuning implementation, Streamlit application, ethics/bias analysis, report writing

8.3 Branch Usage

Branch	Purpose
main	Shared / integration branch — merged output of all member branches
HimasNLP	Member 1's isolated development branch
AfrithNLP	Member 2's isolated development branch
IlmaanNLP	Member 3's isolated development branch

9. Challenges and Lessons Learned

- Missing bio text: the chosen dataset turned out to be fully structured/numeric rather than containing raw biography strings. We addressed this by verbalizing structured features into natural-language profile descriptions, which let every NLP technique in the brief (BoW, TF-IDF, BERT) run on genuine text — a useful lesson in adapting a real dataset to a project's technical requirements rather than assuming the dataset matches the plan exactly.
- Environment setup friction: initial setup issues (running commands from the wrong folder level, PowerShell path confusion) cost time early on — resolved by standardizing on running all commands from the project root and documenting exact steps in the README.
- Editor false positives: TensorFlow's dynamic module loading triggered Pylance import warnings in VS Code despite the code running correctly — a reminder to verify functionality by execution, not by editor linting alone.
- Compute/network constraints for BERT: fine-tuning bert-base-uncased requires downloading ~440MB of weights and is compute-intensive, making it the slowest and most infrastructure-dependent part of the project.

- Model architecture change mid-project: switching Member 2's deep learning model from RNN running training and updating all downstream comparison tables — reinforced the value of keeping the pipeline modular so a single component can be swapped without rewriting the rest.
- Data leakage discipline: care was taken to fit vectorizers (BoW/TF-IDF) and scalers only on training data, then apply (not refit) them to validation/test data, avoiding an easy but subtle source of inflated metrics.

10. Conclusion

This project demonstrates a complete, working pipeline for detecting fake Instagram accounts using NLP-derived and structured profile features. Three independently developed model pairs — Logistic Regression/LSTM, Random Forest/RNN, and SVM/BERT — were built, evaluated with a consistent multi-metric methodology, and compared on a shared held-out test set. Random Forest is currently the top-performing model at 98.7% accuracy and 0.992 ROC-AUC, and has been made accessible through a functional Streamlit web application with a confidence threshold and clear ethical disclaimers.

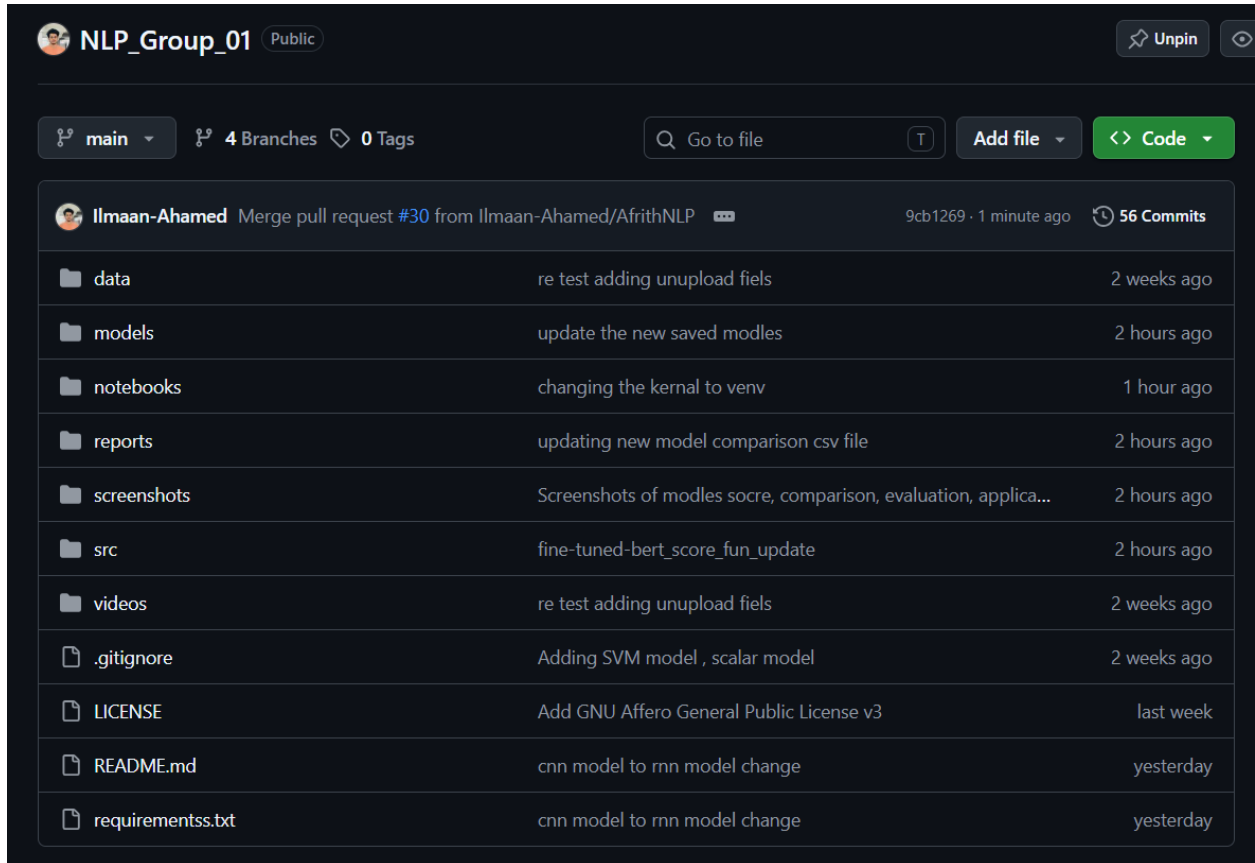
11. References

- [1] free4ever1, "Instagram Fake Spammer Genuine Accounts Dataset," Kaggle, 2019. [Online]. Available: <https://www.kaggle.com/datasets/free4ever1/instagram-fake-spammer-genuine-accounts>
- [2] F. Pedregosa et al., "[Scikit-learn: Machine Learning in Python](#)," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015. [Online]. Available: <https://www.tensorflow.org/>
- [4] S. Hochreiter and J. Schmidhuber, "[Long Short-Term Memory](#)," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [5] L. Breiman, "[Random Forests](#)," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "[BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#)," in *Proc. NAACL-HLT, 2019*, pp. 4171–4186.
- [7] T. Wolf et al., "[HuggingFace's Transformers: State-of-the-Art Natural Language Processing](#)," *arXiv:1910.03771*, 2020.
- [8] S. Bird, E. Klein, and E. Loper, [Natural Language Processing with Python](#). O'Reilly Media, 2009. (NLTK)
- [9] Streamlit Inc., <https://docs.streamlit.io/>
- [10] C. Cortes and V. Vapnik, "[Support-Vector Networks](#)," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

Appendix

A.1 Additional Screenshots

Github Repo: https://github.com/Ilmaan-Ahamed/NLP_Group_01



The screenshot displays the GitHub repository page for **NLP_Group_01** by user **Ilmaan-Ahamed**. The repository is public and has 4 branches and 0 tags. The main branch is selected. The page shows a merge pull request #30 from Ilmaan-Ahamed/AfrithNLP, committed 1 minute ago with 56 commits.

File/Folder	Commit Message	Commit Time
data	re test adding unupload fiels	2 weeks ago
models	update the new saved modles	2 hours ago
notebooks	changing the kernal to venv	1 hour ago
reports	updating new model comparison csv file	2 hours ago
screenshots	Screenshots of modles socre, comparison, evaluation, applica...	2 hours ago
src	fine-tuned-bert_score_fun_update	2 hours ago
videos	re test adding unupload fiels	2 weeks ago
.gitignore	Adding SVM model , scalar model	2 weeks ago
LICENSE	Add GNU Affero General Public License v3	last week
README.md	cnn model to rnn model change	yesterday
requirements.txt	cnn model to rnn model change	yesterday